

Prototype implementation of IC3/PDR with Generalization

Comparison to k -induction

Presented by Lucky M. Kispotta

Advised by MK Srivas

Chennai Mathematical Institute

2026-06-05

Table of Contents

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Content

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Goal

- Understanding the IC3/PDR.
- Develop a prototype for research
- Implement generalization of counterexamples for improving efficiency.
- Compare performance of IC3 with k -induction.
- Compare performance of IC3 with/without generalization.

Key distinguishing features

- IC3 (“Incremental Construction of Inductive Clauses for Indubitable Correctness) algorithm,
is one of the most efficient algorithms which solves the problem of verifying the safety of a Property given a Transition system.
 - *Incremental Construction* : works by building frames gradually.
 - *Inductive Clauses* : Propagate inductive clauses for faster convergence.
 - *Indubitable Correctness* : Prove a property is correct.
- **Advantages over k -induction:** Use only one length transition enrolling per SAT call.

Content

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Inputs/Outputs

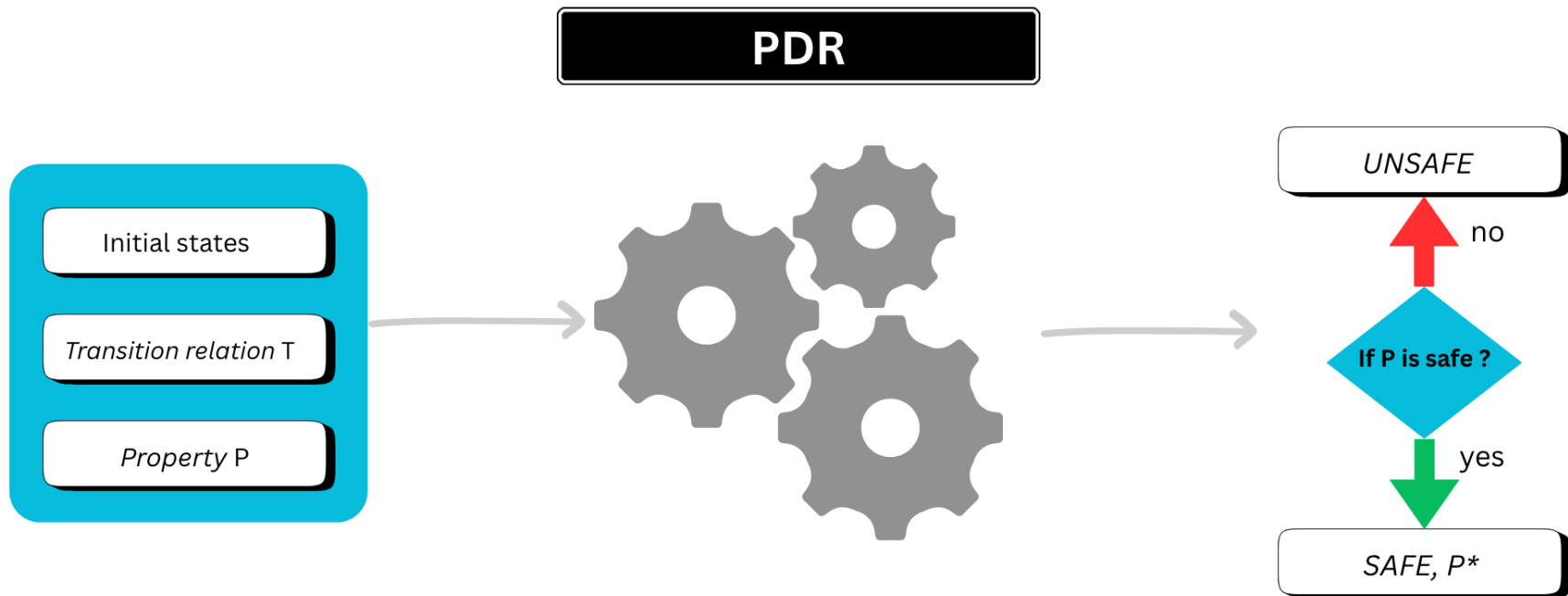


Figure 1: PDR input/outputs

Frames and Algorithmic Invariants

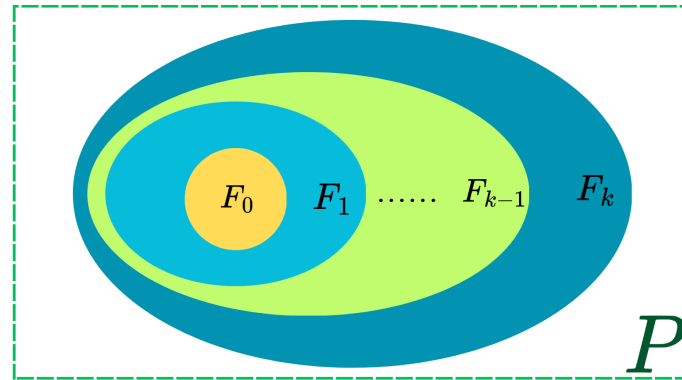


Figure 2: Visualization of frames

The algorithm maintains a set of candidate inductive invariants (Frames):

Algo Inv 1: Frame F_0 contains the set of initial states.

Algo Inv 2: The series of frames is monotone, $F_i \subseteq F_{i+1}$.

Algo Inv 2a): Clauses of F_{i+1} is a subset of clauses of F_i .

Algo Inv 3: None of the frames contain $\neg p$.

Algo Inv 4: Every F_{i+1} is an over-approximation of 1-step image of states in F_i .

Termination: $\exists i, F_i == F_{i+1}$

Blocking Phase : 1) extending frames

- At Frame i , the algorithm tries to extend the set of frames to $i + 1$.
- It initializes frame $i + 1$ with P .

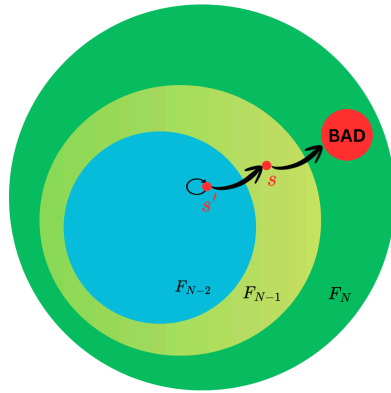


Figure 3: s is the CTI

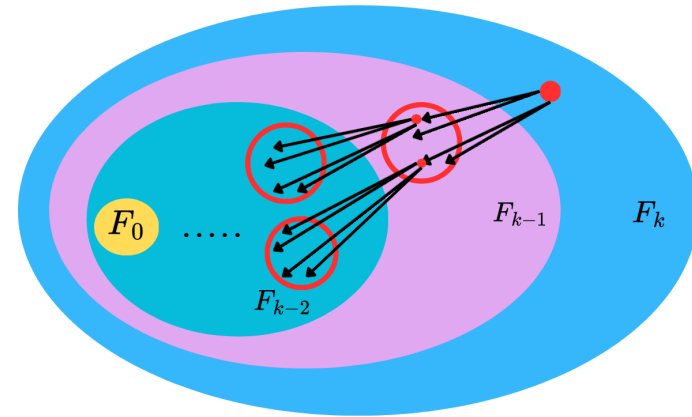


Figure 4: Recursively block CTIs in the previous frames

- The algorithm blocks the CTI at frame i and tries recursively blocking all the predecessor of s in Frame $k - 1$.
- If some predecessor $\in$ Init set then this is a concrete counterexample. And this is a counterexample trace.

Blocking phase: 2) Generating CTI[k]

Checking if a CTI exists at frame k :

$$\text{SAT}[T(x, s') \wedge \neg P(s') \wedge F_k(s)]$$

Check performed to find the predecessor:

$$\text{SAT}[T(x, s') \wedge F_{k-1}(s)]$$

s' is the CTI observed at frame k .

A better SAT check.

$$\text{SAT}[T(x, s') \wedge F_{i-1}(x) \wedge \neg s']$$

This forbids CTIs having self loops from recurring.

Blocking phase: 3) Blocking

- Given a CTI s' at Frame i , it is blocked at frame j , $1 \leq j < i$. That is,
$$\text{UNSAT}[F_j(s) \wedge T(s, s') \wedge \neg s]$$
- If a CTI reaches F_0 before being blocked the CEX $\rightarrow \text{ABORT Unsafe}$.
- Blocking by **STRENGTHENING** (done by adding $\neg s$ to Frame $1 \rightarrow i$).

Propagation Phase using learning inductive clauses

Inductive clause ?

If a clause c is *inductive* at frame i if:

$$\text{VALID}[c_j(s) \wedge T(s, s') \wedge F_i(s) \Rightarrow c_j(s')]$$

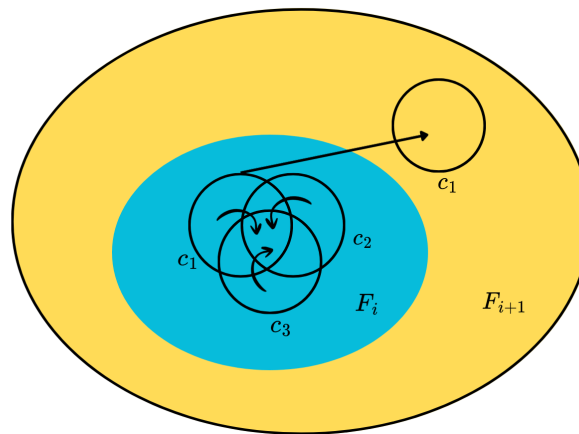


Figure 10: Visualization of propagation phase.

Propagate the inductive clause.

Overview of the algorithm

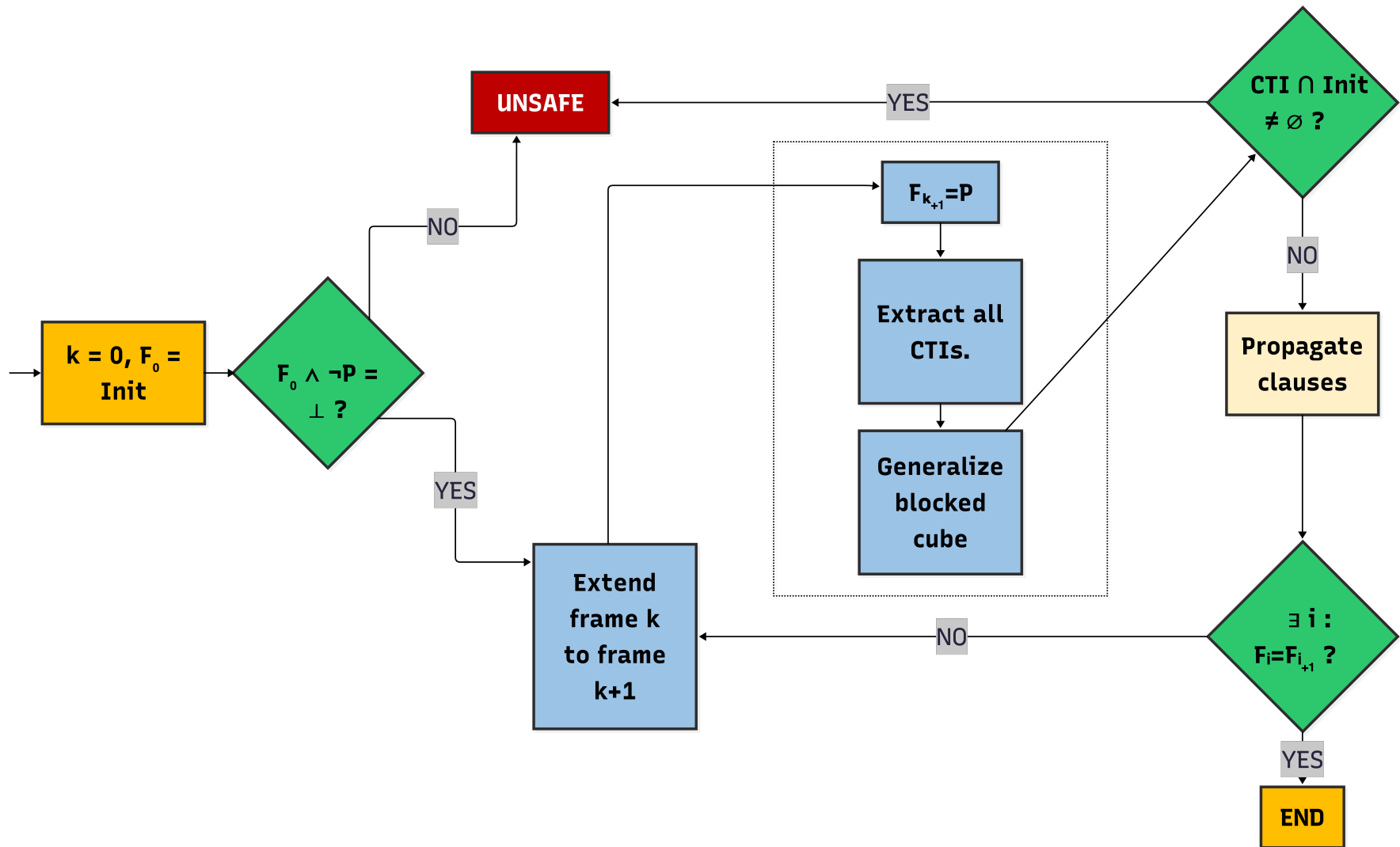


Figure 11: PDR flowchart

Content

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

What is Generalization ?

Generalization in **IC3** is done in two places :

1. Generalizing the CTI.

$$\{x \mid \text{SAT}[T(x, s') \wedge F_{i-1}(x) \wedge \neg s']\}$$

2. Generalizing the blocked clause.

$$\{x \mid \text{UNSAT}[T(x, s') \wedge F_{i-1}(x) \wedge \neg s']\}$$

Generalizing the CTI

Problem

Given a complete CTI cube $c : x_1 \wedge \neg x_2 \wedge x_3$, find the largest set of literals that could be dropped ?

One naive method is to drop literals one by one and check using **SAT** whether the resulting cube is still *blocked* (i.e. whether $\neg c'$ is inductive relative to F_{k-1}). Suppose $c' : x_1$.

$$\text{SAT}[T(x, s') \wedge F_{i-1}(x) \wedge \neg s']$$

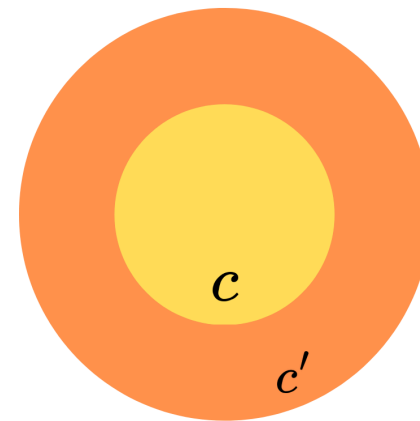


Figure 13: c' has fewer literals than c , $c \subseteq c'$

Ternary Simulation

Given a **CTI** $c : (l_1' = 1)$, find a maximal set of predecessors of c using *Ternary Simulation*.

a SAT query returns predecessor $s : (l_1, l_2, l_3) = (0, 1, 0)$.

As c fixes only l_1 , only

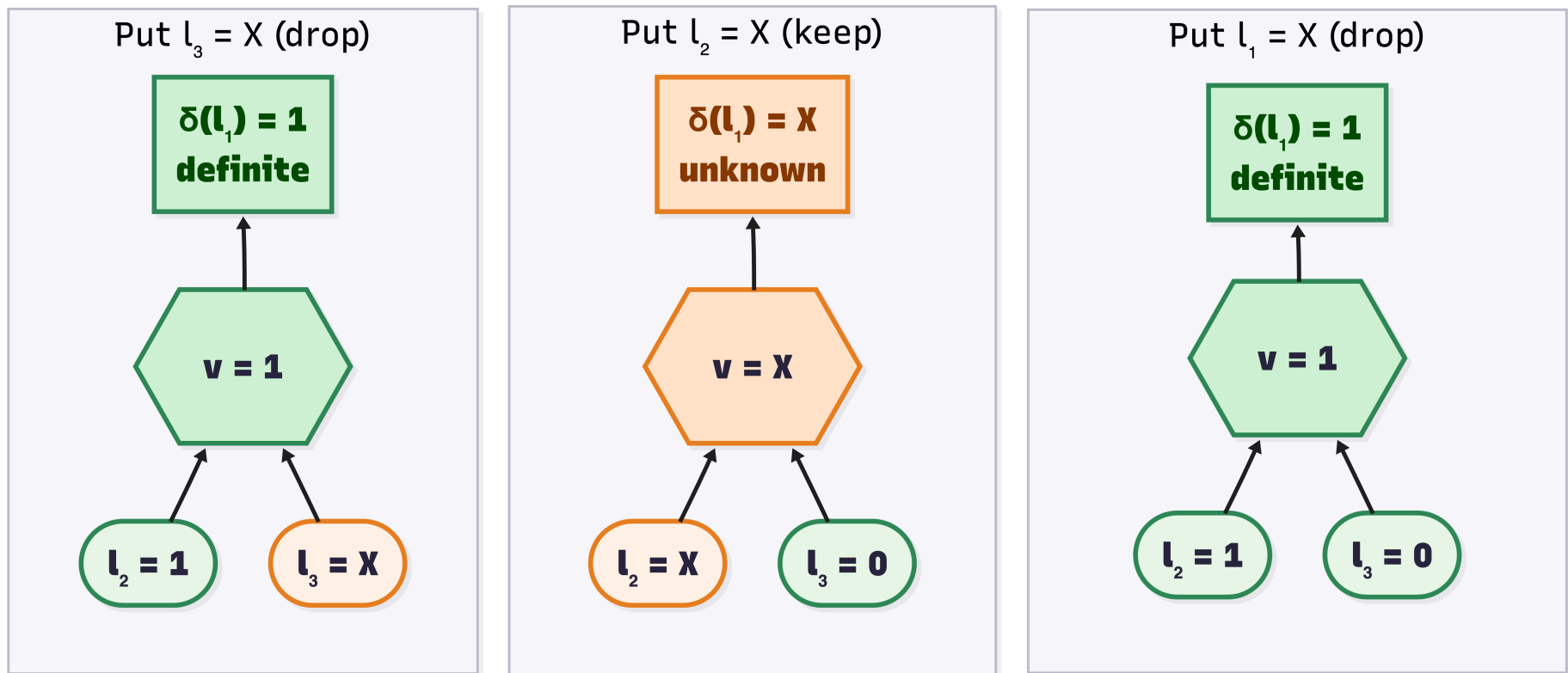
$$\delta(l_1) = l_2 \vee l_3$$

is consulted. We shrink s .

Table 1: Extended AND rules for ternary simulation.

a	b	$a \wedge b$
X	0	0
X	1	X
X	X	X

Ternary Simulation



Generalizing the Blocked Clause

- Idea is to enhance the number of blocked states.
- Use one **UNSAT** check instead.
- Now using *unsat-core* get a subset of *literals* which is sufficient for **UNSAT**.
- Remove respective literals (which do not intersect with Init) from c , shrinks the cube.

Generalizing the Blocked Clause

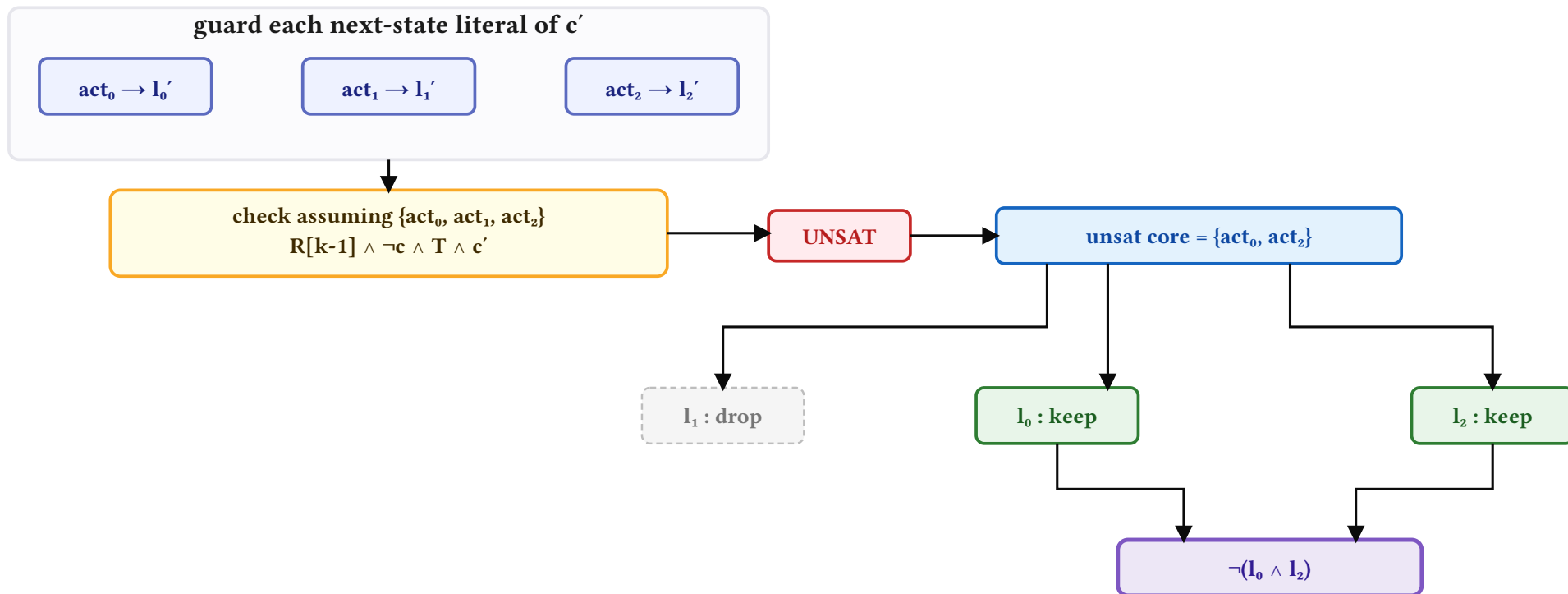


Figure 15: Generalization using *unsat-core*

Content

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Summary of Implementation

- Propositional level safety verification **PDR** algorithm.
- **Delta-encoded frames** : Clauses stored at their highest frame.

$$F_k = \bigwedge_{i \in \mathbb{Z}^+ \text{ and } k \leq i \leq N} \delta_i$$

here, $\delta_k = F_k - F_{k+1}$, $F_0 = \text{Init}$ (base)

- **Priority-queue** proof obligations and iterative blocking phase.
- Implemented **Propagation** and two **generalization** techniques.
- Single incremental z3 solver across the program.
- Environment invariants using sticky latch.

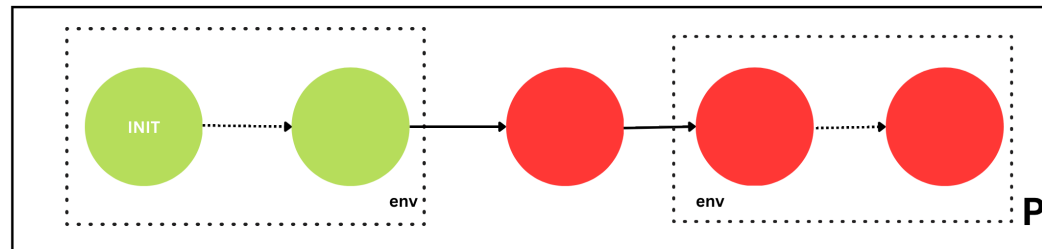


Figure 16: The color shows $\neg\text{viol}$, the negation of the sticky latch viol. viol remembers whether any earlier state on the path violated the environment constraint: it sets to 1 on the first violation and stays set. The property is rewritten to $P = \neg(\text{bad} \wedge \neg\text{viol} \wedge \text{env})$ i.e a bad state is a real violation only if the environment was respected at every previous step ($\neg\text{viol}$) and still holds at that state (env).

Aiger

- The tool accepts input in **AIGER** format.
- Aiger format is popular file format used to represent circuits.
- Contains primary inputs, states, 2 input **AND** gates and a bad property ($\neg P$), reset values (Init).

ICEpie

Implemented the algorithm with the optimizations in a tool called **ICEpie**.

```
(base) luckyfuz@Luckys-MacBook-Pro src % python main.py ./benchmarks/hmcc/ntu/unsat/microban_1.aig
Welcome to ICEpie :)

Run configuration
Setting      Value
-----
Target      ./benchmarks/hmcc/ntu/unsat/microban_1.aig
Max frames  100
Timeout     none
Frame propagation on
Ternary generalization off
Unsat-core generalization off
Max latches 1000

Runtime statistics - microban_1.aig
Metric      Value
-----
Verdict      SAFE
Latches      23
Inputs       23
And gates    278
Constraints  124

Frames built 19
Fixpoint frame 3
Clauses learned 436
Live clauses  138
SAT calls     22856
Ternary minterms 224
Ternary probes 5374
Invariant clauses 56
Invariant literals 268
Runtime       39.039s

SAFE: microban_1.aig :: SAFE : property holds
(base) luckyfuz@Luckys-MacBook-Pro src %
```

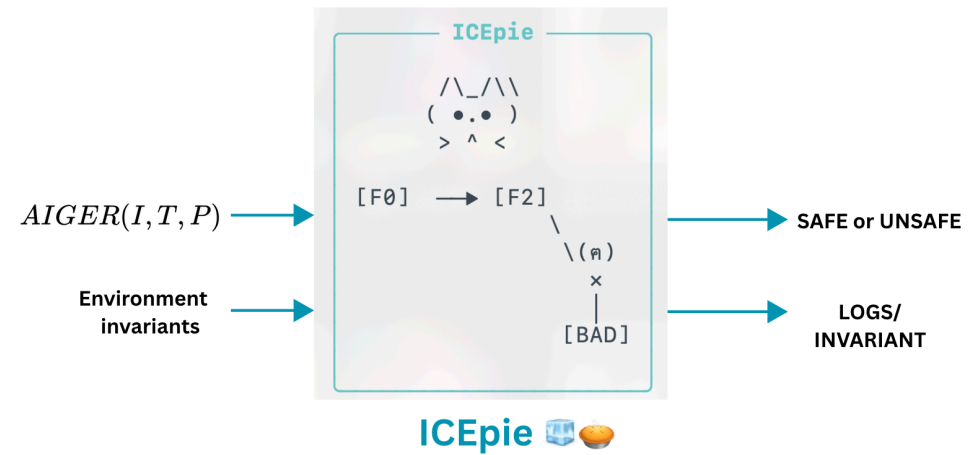


Figure 17: Running examples using the tool

Comparison with k-induction

Benchmark	PDR k	k-ind k	PDR s	k-ind s
x-lte-x-squared-32	1	1	0.06	0.10
traffic-light-prescale-8	2	1	0.005	0.006
tic-tac-toe-3x3	4	2	1.84	0.04
prime-20996011-accumulator	2	1	41.5	2.57
petersons-2-threads-1-core	7	38	0.49	9.48
petersons-2-threads-2-cores	6	44	0.36	12.6
microban_1	23	94	73	4680

Timeout = 40s

Takeaway. k-induction sometimes does better when the property is near-inductive ($k = 0, 1$), but explodes on sequential protocols (Peterson: $k = 37-43$ vs. fixpoint 6–7). Depth $\neq$ time, prime proves at $k = 0$ in 2.6 s while PDR's fixpoint-2 proof costs 41 s.

Benchmarks: tniessen/aiger-safety-properties (T. Niessen)

Comparison of optimization techniques

Benchmark	Best opt	Base (s)	Opt (s)	Speedup	SAT b→o
Generalization helps					
traffic-light-prescale-4 ✓	ternary	0.067	0.004	16.8×	163 → 23
petersons-2t-1core ✓	unsatcore	0.477	0.090	5.3×	931 → 316
peter-sons-2t-2cores ✓	unsatcore	0.498	0.059	8.4×	931 → 208
traffic-light-prescale-8 ✓	ternary	1.387	0.005	277×	2563 → 23
semaphore-2t-limit-1 ✓	unsatcore	timeout	7.504	solves	— → 9930
unary-counter-max-8 ✗	both	0.235	0.011	21.4×	661 → 33

Comparison of optimization techniques

Benchmark	Best opt	Base (s)	Opt (s)	Speedup	SAT b→o
Generalization does not help					
fibonacci-03-0x6 ✓	unsatcore	0.099	0.449	0.2×	276 → 1607
fibonacci-04-0xc ✓	unsatcore	0.504	5.260	0.1×	1068 → 14118
fibonacci-05-0x14 ✓	—	—	timeout	timeout	4188 → —

The generalization fails to optimize when literals cannot be dropped from clauses (most literals matter).

Content

Goal

IC3/PDR and Key Ideas

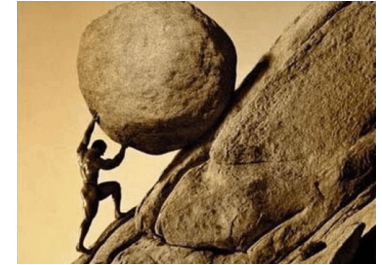
Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Summer Project



- Rewrite the engine in C++ for performance enhancement.
- Replace the Z3 backend with a dedicated incremental SAT solver (CaDiCaL via PySAT) [*experimental*]
- Add a new generalization technique based on “Better Generalization in IC3”
- Add more **LTL** capabilities to the engine. Currently something similar to $G \neg \text{bad}$ is being done.
- *Portfolio mode* - run **PDR** and ***k*-indunction** concurrently. Return first one to solve.

Content

Goal

IC3/PDR and Key Ideas

Generalization

ICEpie / Implementation

Summer Project

Literature Survey

Some works extending **IC3**:

Baseline: propositional **safety** checking [1], [2].

Richer theories (SMT-based)

- Implicit predicate abstraction: LRA / EUF [3]
- GPDR & **Spacer** : Horn clauses + arithmetic (in Z3) [4], [5]
- **AVR** : word-level / bit-vector RTL [6]

Beyond safety (temporal logic)

- **FAIR** : IC3-based fair-cycle detection [7]
- **k-Liveness** : liveness as counting $\rightarrow$ safety [8]
- **LTL** via k-Liveness + implicit abstraction [9]

Theory + temporal can be **stacked**: IC3-IA + k-Liveness $\rightarrow$ LTL over infinite-state arithmetic [9].

References

- [1] A. R. Bradley, “SAT-Based Model Checking Without Unrolling,” in *VM-CAI*, 2011.
- [2] Eén, Niklas, Mishchenko, Alan, Brayton, and Robert K., “Efficient Implementation of Property Directed Reachability,” in *FMCAD*, 2011.
- [3] Cimatti *et al.*, “IC3 Modulo Theories via Implicit Predicate Abstraction,” in *TACAS*, 2014.
- [4] Hoder, Kryštof, Bjørner, and Nikolaj, “Generalized Property Directed Reachability,” in *SAT*, 2012.
- [5] Komuravelli, Anvesh, Gurfinkel, Arie, Chaki, and Sagar, “SMT-Based Model Checking for Recursive Programs,” in *CAV*, 2014.
- [6] Goel, Aman, Sakallah, and Karem A., “AVR: Abstractly Verifying Reachability,” in *TACAS*, 2020.

- [7] Bradley *et al.*, “An Incremental Approach to Model Checking Progress Properties,” in *FMCAD*, 2011.
- [8] Claessen, Koen, Sörensson, and Niklas, “A Liveness Checking Algorithm that Counts,” in *FMCAD*, 2012.
- [9] Cimatti *et al.*, “Verifying LTL Properties of Hybrid Systems with K-Liveness,” in *CAV*, 2014.

Thanks for Listening.

Bye Bye

